

Reviewer Pack — Sparsest-Cut LP Value Lower-Bounds Tseitin Resolution Length

Entry 4d59152d4522 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 05:56:11 UTC

Sparsest-Cut LP Value Lower-Bounds Tseitin Resolution Length

Entry ID: 4d59152d4522 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 05:56:11 UTC

1. Conjecture

Field A (mathematical branch): Leighton-Rao sparsest-cut LP relaxation and metric (semimetric) embedding theory; multicommodity flow LP duality (Leighton-Rao 1988, Linial-London-Rabinovich 1995) — a flow/metric certificate of expansion that is computationally distinct from λ_2 and rarely linked to proof complexity **Field B** (complexity object): Resolution refutation length $L_R(\text{Tseitin}(G, \sigma))$ of odd-charged Tseitin CNFs on 3-regular graphs G (the canonical sub-Frege gateway target for EF lower bounds)

Statement:

For every connected 3-regular graph G on n vertices and odd parity vector $\sigma \in \{0,1\}^V$ with $\sum \sigma \equiv 1 \pmod{2}$, define $\nu(G) := n^2 \cdot \Phi_{LP}(G)$, where $\Phi_{LP}(G) := \min \{ \sum_{(u,v) \in E} d(u,v) : d \text{ a semimetric on } V \text{ with } \sum_{u < v} d(u,v) = 1 \}$ is the Leighton-Rao sparsest-cut LP value (a polynomial-size LP). Then there is an absolute constant $c > 0$ with $\log_2 L_R(\text{Tseitin}(G, \sigma)) \geq c \cdot \nu(G)$ for every such (G, σ) , and moreover $\nu(G) \leq 8$ whenever G has tree-width ≤ 3 (so ν is $O(1)$ on every non-expander family).

Rationale (proposer's reasoning):

Tseitin Resolution lower bounds traditionally invoke spectral expansion, but the LP-dual sparsest-cut certificate is a structurally different — flow-theoretic — witness of expansion that decouples from the second Laplacian eigenvalue and survives spectral perturbations. A direct quantitative link from Φ_{LP} to Resolution length would import the Leighton-Rao / metric-embedding toolkit into proof complexity and give a poly-computable, integrality-gap-aware lower-bound certificate. The cross sparsest-cut-LP $\times$ Tseitin-Resolution has, to my knowledge, fewer than five papers despite both fields being well developed.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d3beb6ce47ef2fa3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 5 sizes $n \in \{8, 10, 12, 14, 16\} \times 3$ families $\times 30$ seeds = 450 trials, the conjecture is SUPPORTED iff (i) $\log_2(T) \geq \nu/40 - 2$ holds in $\geq 80\%$ of trials (mean over seeds per cell), and (ii) $\nu \leq 8$ on every Möbius-ladder and dumbbell trial ($tw \leq 3$ families).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.97	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin resolution lower bound expansion sparsest cut - Leighton-Rao LP relaxation proof complexity resolution length - semimetric embedding multicommodity flow Tseitin formula

Top relevant hits considered: - [s2:10.1007/978-3-642-38536-0_14] Graph Expansion, Tseitin Formulas and Resolution Proofs for CSP - [s2:1b3c2faed501856fd5b542e3b6f6185f210f90be] Expander Flows and Sparsest Cut Linear Programming

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below pivot
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n + 1):
                A[j][k] -= factor * A[i][k]

    # Back-substitute to get solution
    x = [0] * n
```

```

    for i in range(n - 1, -1, -1):
        x[i] = A[i][n] / A[i][i]
        for j in range(i + 1, n):
            x[i] -= A[i][j] * x[j]

    return x

def solve_linear_system(A, b):
    A_aug = [row + [b[i]] for i, row in enumerate(A)]
    solution = gaussian_elimination(A_aug)
    return solution

def generate_random_3regular_graph(n):
    edges = set()
    while len(edges) < n * 3 // 2:
        u, v = random.sample(range(n), 2)
        if (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))
    return list(edges)

def generate_dumbbell_graph():
    K4_edges = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3)]
    bridge_edges = [(4, 5), (4, 6), (4, 7), (5, 8), (5, 9), (5, 10), (6, 8), (6, 9), (6, 10), (7,
    return K4_edges + bridge_edges

def generate_möbius_ladder_graph():
    n = 12
    edges = [(i, (i + 1) % n) for i in range(n)] + [(i, (i + n // 2) % n) for i in range(n)]
    return edges

def tseitin_formula(G, sigma):
    n = len(sigma)
    literals = {u: f"v{u}" for u in range(n)}
    neg_literals = {u: f"~v{u}" for u in range(n)}

    clauses = []
    for u in range(n):
        if sigma[u] == 1:
            clauses.append([literals[u]])
        else:
            clauses.append([-literals[u]])

    for (u, v) in G:
        clauses.append([neg_literals[u], literals[v]])
        clauses.append([neg_literals[v], literals[u]])

    return clauses

def dpll(clauses):
    def search(model):
        if not clauses:
            return model
        unit_clauses = [c[0] for c in clauses if len(c) == 1]
        pure_literals = defaultdict(int)
        for c in clauses:
            for l in c:

```

```

        pure_literals[l] += 1

    if unit_clauses:
        literal = unit_clauses[0]
        new_model = model + [literal]
        new_clauses = [[l for l in c if l != literal and l != -literal] for c in clauses]
        return search(new_model)

    if pure_literals:
        literal, count = max(pure_literals.items(), key=lambda x: abs(x[1]))
        new_model = model + [literal] if count > 0 else model + [-literal]
        new_clauses = [[l for l in c if l != literal and l != -literal] for c in clauses]
        return search(new_model)

    literal = next(iter(model))
    new_model_true = model + [literal]
    new_clauses_true = [[l for l in c if l != literal and l != -literal] for c in clauses]
    result_true = search(new_model_true)
    if result_true:
        return result_true

    new_model_false = model + [-literal]
    new_clauses_false = [[l for l in c if l != literal and l != -literal] for c in clauses]
    result_false = search(new_model_false)
    return result_false

return search([])

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14, 16]
    results = []

    for n in n_values:
        for _ in range(30):
            if n == 8:
                G = generate_random_3regular_graph(n)
            elif n == 10:
                G = generate_dumbbell_graph()
            elif n == 12 or n == 14 or n == 16:
                G = generate_möbius_ladder_graph()
            else:
                raise ValueError("Invalid n value")

            sigma = [random.randint(0, 1) for _ in range(n)]
            if sum(sigma) % 2 != 1:
                continue

            phi_lp = float('inf')
            for d in range(1, n + 1):
                A = [[0] * (n * (n - 1) // 2) for _ in range(n * (n - 1) // 2)]
                b = [0] * (n * (n - 1) // 2)
                for i, u in enumerate(range(n)):
                    for j, v in enumerate(range(u + 1, n)):
                        if (u, v) in G:

```

```

        A[i * (n - 1) // 2 + j][i * (n - 1) // 2 + j] = d
        b[i * (n - 1) // 2 + j] += 1 / n
    try:
        x = solve_linear_system(A, b)
        phi_lp = min(phi_lp, sum(x))
    except Exception as e:
        print(f"Error solving linear system for n={n}: {e}")

nu = n * n * phi_lp

clauses = tseitin_formula(G, sigma)
T = len(dpll(clauses))

results.append({
    "metric_name": "nu",
    "metric_value": nu,
    "instances_tested": 1,
    "conjecture_holds": log2(T) >= nu / 40 - 2,
    "counterexample": "" if log2(T) >= nu / 40 - 2 else f"n={n}, T={T}, nu={nu}"
})

mean_nu = sum(result["metric_value"] for result in results) / len(results)
std_nu = math.sqrt(sum((result["metric_value"] - mean_nu) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_nu": mean_nu,
    "std_nu": std_nu,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_nu = sum(result["mean_nu"] for result in results) / len(results)
    std_nu = math.sqrt(sum((result["mean_nu"] - mean_nu) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.8) / len(results)

    if all(result["support_fraction"] >= 0.8 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_nu} std={std_nu} support_fraction={support_fraction}")
    elif any(result["counterexample"] != "" for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
Error solving linear system for n=8: division by zero
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71laf89a.py", line 185, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71laf89a.py", line 157, in run_trial
    clauses = tseitin_formula(G, sigma)
              ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71laf89a.py", line 75, in tseitin_formula
    clauses.append([-literals[u]])
                   ^^^^^^^^^^^^^
```

TypeError: bad operand type for unary -: 'str'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `tseitin_formula` (unary minus on a string literal) before any trials completed, so neither the support nor falsification criteria could be evaluated. No data was produced. | next: Fix the literal-encoding bug (ensure `literals[u]` is an int, not a str) in `tseitin_formula` and re-run; also handle the `n=8` ‘division by zero’ in the LP solver (likely degenerate semimetric normalization).

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	329152
2	preregistration	claude_max	opus	0	0	7559
3	novelty	claude_max	opus	0	0	3230
4	novelty	claude_max	opus	0	0	9794

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	77613
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	173246
7	judge	claude_max	opus	0	0	4755

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 605350 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/4d59152d4522.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4d59152d4522.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4d59152d4522.tar.gz (if generated)